

赛前速查册

题目归档与标签板子

题目	08
标签板子	05
生成日期	2026-09-06

板子部分按标签顺序排在前面，做题记录按完成时间倒序排在后面。内容来自个人网站，保留原有思路与表述。

CONTENTS

目录

目录	2
标签板子	3
状压 dp	3
子集枚举	3
容斥划分	3
SOS dp	3
就是超集和、子集和	3
板子	3
凸包	4
Andrew 凸包	4
板子	4
回文字符串Manacher	4
作用	4
大致思路	4
板子	4
位运算	5
对位运算的定义后面慢慢补充	5
做题记录	6
兜风	6
图穷匕首突起	7
天开武道会	9
天使领域破解	11
悬赏千金抓人	12
聂政认尸殉弟	13
评测排序	15
字典最小博弈	17
上场前最后检查	20

标签板子

从识别标签进入对应知识点；公式、变量和代码已统一为适合屏幕阅读与打印的格式。

01 / state-compression-dp

状压 dp

用一个整数表示集合状态，在可控的指数复杂度内完成动态规划。

子集枚举

```
for (int T = S; T; T = (T - 1) & S)
    ...
```

容斥划分

按照二进制中 1 的个数决定正负。

02 / sos-dp

SOS dp

已知集合的价值，计算所有超集的价值之和。

就是超集和、子集和

已知集合的价值，计算所有超集的价值之和。

设 $dp[S][i]$ 表示集合 S 考虑前 i 位时，值为 0 的位可以自由变化后得到的价值和。

当第 i 位是 0 时：

$$dp[S][i] = dp[S][i-1] + dp[S|(1 \ll (i-1))][i-1]$$

比如 $dp[0010][3] = dp[0110][2] + dp[0010][2]$ 。

子集和同理，把 1 看作 0。

板子

```
for (int bit = 0; bit < m; bit++)
    for (int s = 0; s < (1 << m); s++)
        if (!(s & (1 << bit)))
            dp[s] = dp[s] + dp[s | (1 << bit)];
```

这里原本 $dp[s]$ 是集合 s 的价值，通过维度空间压缩得到了超集和。

03 / 凸包

凸包

来回扫两遍，左右转只需要保持一致即可。

Andrew 凸包

来回扫两遍，左右转只需要保持一致即可。

板子

```
sort(points.begin(), points.end());

vector<Point> stk;

for (auto p : points)
{
    while (stk.size() >= 2 &&
           cross(stk.back() - stk[stk.size() - 2],
                p - stk.back()) <= 0)
        stk.pop_back();

    stk.push_back(p);
}

int lower_size = stk.size();

for (int i = n - 2; i >= 0; i--)
{
    auto p = points[i];

    while (stk.size() > lower_size &&
           cross(stk.back() - stk[stk.size() - 2],
                p - stk.back()) <= 0)
        stk.pop_back();

    stk.push_back(p);
}

stk.pop_back();
// 注：这里最后得到的就是凸包顺序。如果 A、B、C 共线，
// 那么 B 会被排除，只得到极点；如果要保留 B，则需要额外处理 cross == 0。
```

04 / 回文字符串Manacher

回文字符串Manacher

可以得到一个字符串每个位置的最大回文半径， $O(n)$ 复杂度。

作用

可以得到一个字符串每个位置的最大回文半径，复杂度为 $O(n)$ 。

大致思路

维护端点最靠右的回文串 `center` 和 `right`。当前处理 `i`，`i` 关于 `center` 的对称点 `j` 的回文半径是已知的，回文的信息也可以搬过来。

板子

```
vector<int> manacher(const string& s) {
    string t;
    t.reserve(2 * s.size() + 1);
    for (char c : s) {
        t.push_back('#');
        t.push_back(c);
    }
    t.push_back('#');
    int m = t.size();
    vector<int> p(m);

    int center = 0;
    int right = 0; // 当前回文右端点的后一位

    for (int i = 0; i < m; i++) {
        if (i < right) {
            int mirror = 2 * center - i;
            p[i] = min(p[mirror], right - i);
        } else {
            p[i] = 1;
        }

        while (i - p[i] >= 0 &&
               i + p[i] < m &&
               t[i - p[i]] == t[i + p[i]]) {
            p[i]++;
        }

        if (i + p[i] > right) {
            center = i;
            right = i + p[i];
        }
    }

    return p;
}
```

05 / 位运算

位运算

关于牛客8/7场的A题，里面涉及一个贪心构造：给定一个ans，里面为1的位是可以操作的，给定一个数组，问随意操作已知的位，能不能构造出单调不递降...

对位运算的定义后面慢慢补充

关于牛客 8/7 场的 A 题，里面涉及一个贪心构造：给定一个 ans，里面为 1 的位是可以操作的；给定一个数组，问随意操作已知的位，能不能构造出单调不递降的数组？

核心就是 `getv(int last, int now)`：从 ans 的最高位开始遍历每一位 1，考虑什么时候这一位必须是 1。当后面的所有可操作位都是 1 还不够时，这一位一定是 1。

涉及到寻找最高位的 1：

```
63 - __builtin_clzll(x)
```

这里给出 `builtin` 的一系列用法：

```
__builtin_popcountll(x); // population count
__builtin_clzll(x);      // count leading zeros
__builtin_ctzll(x);      // count trailing zeros
__builtin_parityll(x);   // 判断 1 的个数的奇偶
```

PART 02 / 8 题

做题记录

每题保留题目大意、原始解题思路和提交代码，便于比赛前按标签回忆完整路径。

01 / 20260901-01

兜风

牛客 2026-09-01 点双连通分量 代码: 20260901-01.cpp

题目大意

一次兜风就是从当前点出发，至少经过3个点，不经过重复点（除了当前点），最后回到当前点，问每个点最多能进行多少次兜风。

思路

有一个结论，如果当前点是 1，那么设它的邻居有 2, 3, 4, 5, 6。假设存在一个环经过 1, 2, 3，另外也有环经过 1, 3, 4 和 1, 4, 5，那么 2, 3, 4, 5 认为是一组可以两两搭配的集合，可以证明一定两两搭配。

可以看出，就是求点双。

提交代码

```
#include<bits/stdc++.h>
#define int long long
using namespace std;
const int N=2e5+7;
vector<pair<int,int> > g[N];
int dfn[N],low[N],f[N];
int tim;
int bianhao;
stack<int> stk;

void tarjan(int fa,int u)
{
    low[u]=dfn[u]=++tim;
    for(auto it:g[u])
    {
        int v=it.first;
        if(v==fa)continue;

        if(!dfn[v])
        {
            stk.push(it.second);
            tarjan(u,v);
            low[u]=min(low[u],low[v]);

            if(low[v]>=dfn[u])
            {
                while(1)
                {
                    int i=stk.top();
                    stk.pop();
                    f[i]=bianhao;
                    if(i==it.second)break;
                }
                bianhao++;
            }
        }
        else if(dfn[v]<dfn[u])
        {
            stk.push(it.second);
```

```

low[u]=min(low[u],dfn[v]);
}
}
}

void solve()
{
    int n,m;
    cin>>n>>m;
    tim=0;
    bianhao=0;
    while(!stk.empty())stk.pop();
    for(int i=0;i<=n;i++)
    {
        g[i].clear();
        dfn[i]=low[i]=f[i]=0;
        bianhao=1;
    }
    for(int i=1;i<=m;i++)
    {
        int u,v;
        cin>>u>>v;
        g[u].push_back({v,i});
        g[v].push_back({u,i});
    }
    for(int i=1;i<=n;i++)
    {
        if(dfn[i]==0)tarjan(0,i);
    }
    for(int i=1;i<=n;i++)
    {
        int ans=0;
        map<int,int> mp;
        for(auto it:g[i])
        {
            mp[f[it.second]]++;
        }
        for(auto it:mp)
        {
            ans+=it.second/2;
        }
        cout<<ans<<" \n"[i==n];
    }
}

signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    int T;
    cin>>T;
    while(T--)
    {
        solve();
    }
}

```

02 / 20260810-01

图穷匕首突起

码蹄集 2026-08-10 概率论 代码: 20260810-01.cpp

题目大意

给定 n 根木棍的长度，小球每次落在某一根木棍的概率是 l_i/sum 。问，第一次连续三次落在同一根木棍的次数期望是多少？

思路

这是一个纯概率论问题。关键是状态要定义好+会写期望转移式。

定义期望： E_0 表示还没有开始， $E_{1,i}$ 表示最后连续落在第 i 根木棍一次， $E_{2,i}$ 表示两次。

期望转移式：

$$E_0 = 1 + \sum(i) p_i E_{1,i}$$

$$E_{1,i} = 1 + p_i E_{2,i} + \sum(j \neq i) p_j E_{1,j}$$

$$E_{2,i} = 1 + \sum(j \neq i) p_j E_{1,j}$$

关键就是前面有一个 1，然后按发生概率转移。

这里的 $\sum(j \neq i) p_j E_{1,j}$ 表示 $1 \leq j \leq n$ 且 $j \neq i$ 。

解方程，就是消元，然后同乘 p_i 累加 $E_{1,i}$ 。

提交代码

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define double long double
const int mod=1e9+7;
const int N=5e5+7;
int l[N];

int qpow(int a,int p)
{
    int ans=1;
    while(p)
    {
        if(p&1)ans=ans*a%mod;
        a=a*a%mod;
        p>>=1;
    }
    return ans;
}

int inv(int a){
    return qpow(a,mod-2);
}

signed main()
{
    //cout<<log10(0ull-1ull);
    int n;
    cin>>n;
    int sum=0;
    for(int i=1;i<=n;i++)
    {
        cin>>l[i];
        sum+=l[i];
    }
    int cnt=0;
    for(int i=1;i<=n;i++)
    {
        int p=l[i]*inv(sum)%mod;
        int tmpa=p*p%mod;
        int tmpb=(1+p+tmpa)%mod;
        tmpa=tmpa*p%mod;
        cnt+=tmpa*inv(tmpb)%mod;
        cnt%=mod;
    }
    cout<<inv(cnt);
}
```


03 / 20260808-02

天开武道会

牛客 2026-08-08 贪心 代码: 20260808-02.cpp

题目大意

给定两个排列 ($n \leq 2e5$)。给定 x ，希望 x 能成为冠军。每次从这两个排列的左边选择第一个没有被淘汰的人，进行对战。如果选出同一个人，则发生死锁。如果一直没有死锁，最后剩下那个人就是冠军。如果 x 能成为冠军，问求淘汰顺序？

思路

这个题比较特别，它的贪心很有意思。

先求必要条件：

1. 对于 $1 \leq i \leq n$ ，如果两个排列前缀集合相同，那么一定会死锁。
2. 如果存在一个数，在两个排列中都出现在 x 后面，那么 x 一定不是冠军。

然后直接贪心构造：

设排列是 P, Q ，如果当前是 u 对战 v ，那么看 u 在 Q 、 v 在 P 里面的排名，选择排名靠前的淘汰。

比如当前队列是 $u, \dots, v$ 和 $v, \dots, u$ ，那么 v 的排名更靠前，淘汰 v 变成： $u, \dots$ 和 $\dots, u$ 。这个时候，因为我们继承了必要条件：没有相同的真前缀，继续检查发生变化的前缀。考虑第二个队列 $\dots, u$ ，如果没有到 u ，那一定不行；到了 u ，和原本就相差了 v ，也不会有相同的真前缀。

但是如果选择淘汰 u ：检查变化的区间， $\dots, v$ 和 $v, \dots$ 有可能发生相同前缀。

所以这个题就是：提出必要条件，贪心维护必要条件，最终变成充分条件。

但是这里看的是 u, v 的实时排名，显然不好维护。交了一发静态排名的，居然过了。

下面证明，如果有静态排名 $u < v$ ，则动态排名 $u \leq v$ ：

假设队列 1 是 $4, 5, 6, u, 1, 2, 3, v$ ，队列 2 是 $1, 2, 3, v, 4, 5, 6, u$ 。 $4, 5, 6$ 是队列 1 中被淘汰的， $1, 2, 3$ 同理。可以证明它们的位置肯定都在 u, v 中间（根据我们的贪心构造方法），所以不可能造成方向翻转。

提交代码

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define double long double
const int N=2e5+7;
int a[N],b[N];
int isv[N];
vector<int> ans;
int mpa[N];
int mpb[N];
int n,x;

int check()
{
    priority_queue<int,vector<int> > qa;
    priority_queue<int,vector<int> > qb;
    for(int i=1;i<n;i++)
    {
        qa.push(a[i]);
        qb.push(b[i]);
        while(!qa.empty()&&qa.top()==qb.top())
```

```

{
//cout<<qa.top()<<' '<<qb.top()<<endl;
qa.pop();
qb.pop();
}
if(qa.empty())return false;
}
//cout<<"??";
for(int i=1;i<=n;i++)
{
if(a[i]==x)
{
i++;
while(i<=n)isv[a[i++]]=1;
}
}
for(int i=1;i<=n;i++)
{
if(b[i]==x)
{
i++;
while(i<=n)
{
if(isv[b[i++]==1)return false;
}
}
}
return true;
}

signed main()
{
cin>>n>>x;
for(int i=1;i<=n;i++)
{
cin>>a[i];
mpb[a[i]]=i;
}
for(int i=1;i<=n;i++)
{
cin>>b[i];
mpa[b[i]]=i;
}
if(!check())
{
cout<<"NO";
return 0;
}
for(int i=1;i<=n;i++)isv[i]=0;
int pa=1,pb=1;
while(a[pa]!=x||b[pb]!=x)
{
if(a[pa]==x)
{
ans.push_back(b[pb]);
isv[b[pb]]=1;
}
else if(b[pb]==x)
{
ans.push_back(a[pa]);
isv[a[pa]]=1;
}
else
{
if(mpa[a[pa]]<mpb[b[pb]])
{
ans.push_back(a[pa]);
isv[a[pa]]=1;
}
else
{
ans.push_back(b[pb]);
isv[b[pb]]=1;
}
}
while(pa<=n&&isv[a[pa]])pa++;
while(pb<=n&&isv[b[pb]])pb++;
}
int sz=ans.size();

```

```
cout<<"YES\n";
for(int i=0;i<sz;i++)
{
    if(i)cout<<' ';
    cout<<ans[i];
}
}
```

04 / 20260808-01

天使领域破解

牛客 2026-08-08 位运算 代码: 20260808-01.cpp

题目大意

给定 n 个数 ($2e5$)，可以进行若干次操作，每次可以选择一个 k ，选择任意个元素和 k 做异或，使得整个数组单调不下降，求最终 k 的总和最小值。

思路

显然很容易可以发现，必须要按位考虑，而且 k 每次最好只有一位是 1。

然后按最高位考虑，希望迫不得已才选择操作这一位。

什么时候是必须操作的？一开始认为这是一个神秘的区间分割、贪心分配，比如第 4 位是 0, 0, 1, 1, 1，那么只需要考虑两个子区间；如果某一位选择操作，那么每个子区间可以再任意分割成子区间。这显然很复杂，很多时候位运算不会涉及到这种。

其实目前的问题是：“如何判断当前位一定要操作？”其实可以贪心构造，每个位置选择恰好大于上一个数的最小值。

提交代码

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define double long double
const int N=2e5+7;
int a[N];
int ans;

inline int get(int last,int now)
{
    //cout<<last<<' '<<now<<' ';
    int k=ans;
    while(k!=0)
    {
        int mask=63 - __builtin_clzll(k);
        mask=1ll<<mask;
        k=k^mask;
        if((now|k)<last)now|=mask;
    }
    if(now<last)now+=1;
    //cout<<now<<endl;
    return now;
}

void solve()
{
    int n;
    cin>>n;
    ans=0;
    for(int i=1;i<=n;i++)cin>>a[i];
    for(int i=30;i>=0;i--)
    {
        int last=0;
```

```

int mask=(1<<i)-1;
mask|=ans;
mask=~mask;
//cout<<"i"<<' '<<bitset<10>(mask)<<endl;
for(int j=1;j<=n;j++)
{
    int now=a[j]&mask;
    now=get(last,now);
    //cout<<i<<' '<<now<<endl;
    if(now==1)
    {
        ans=ans|(1<<i);
        break;
    }
    last=now;
}
}
cout<<ans<<'\\n';
}

signed main()
{
    int t;
    cin>>t;
    while(t--)
    {
        solve();
    }
}

```

05 / 20260803-02

悬赏千金抓人

码蹄集 2026-08-03 单调栈 代码: 20260803-02.cpp

题目大意

给定 $n*m$ 矩阵，每个位置给定价值。给定 k ，求总和 $\geq k$ 的矩形最小面积（一个位置面积是1）。

思路

显然直接暴力是 $O(n^4)$ ，会爆，而这个题给的是 $O(n^3)$ 不会爆。希望枚举上下行，然后子问题转化为求和满足 $\geq k$ 的最短子数组。

对于这个子问题，观察前缀和 pre ：如果是 3, 1, 6，那么我们不用考虑 3。假设 3 和后面的 6 构成 $6-3 \geq k$ ，但是 3 前面有更小的 1，则 $6-1 \geq k$ 一定成立，且长度更短。于是可以维护单调的左端点。

但是枚举完左端点，直接继续枚举右端点， $O(n^2)$ 还是炸。于是继续观察，发现：如果当前队列是 1, 2, 3，当前点是 4，假如 1, 4 满足了，那么后面的所有右端点都不需要考虑，因为不可能有更优的答案。

总的来说就是两个小思维点。

提交代码

```

#include <bits/stdc++.h>
using namespace std;
#define int long long
#define double long double
const int N=307;
int a[N][N];
int pr[N][N];
int pre[N];
int n,m,k;

int solve(int len)

```

```

{

int ans=1e18;
deque<int> dq;
dq.push_back(0);
for(int i=1;i<=m;i++)
{
while(!dq.empty()&&pre[i]<=pre[dq.back()])dq.pop_back();
while(!dq.empty()&&pre[i]-pre[dq.front()]>=k)
{
ans=min(ans,(i-dq.front())*len);
dq.pop_front();
}
dq.push_back(i);
}
return ans;
}

void work()
{
cin>>n>>m>>k;
for(int i=1;i<=n;i++)
for(int j=1;j<=m;j++)
cin>>a[i][j];
for(int i=1;i<=n;i++)
{
for(int j=1;j<=m;j++)
{
pr[i][j]=pr[i-1][j]+pr[i][j-1]-pr[i-1][j-1]+a[i][j];
}
}
int ans=1e18;
for(int i=1;i<=n;i++)
{
for(int j=i;j<=n;j++)
{
for(int k=1;k<=m;k++)
{
pre[k]=pr[j][k]-pr[i-1][k];
}
//if(j==i+1)
//{
//cout<<"i:"<<i<<endl;
//for(int k=1;k<=m;k++)
//{
//cout<<pre[k]<<" \n"[k==m];
//}
//}
ans=min(solve(j-i+1),ans);
}
}
if(ans==1e18)cout<<-1<<\n';
else cout<<ans<<\n';
}

signed main()
{
int t;
cin>>t;
while(t-->)work();
}

```

06 / 20260803-01

聂菽认尸殉弟

码蹄集 2026-08-03 并查集 / 离线查询 代码: 20260803-01.cpp

题目大意

给定 n 个点， m 条边，每个点有访问阈值 a_i 。有 q 次查询，每次给出 u, x ，表示从 u 出发，能力为 x 能访问多少个点？（即使 $a_u > x$ 也能访问初始点 u ）

思路

显然普通的 BFS、DFS 不能处理。很多图论的题，并查集还是很管用的。需要配合离线查询，按照访问能力 x 排序，这样能访问的点是只增不减的，于是可以维护并查集，表示当前访问能力能处理什么样的集合。

代码里面，需要注意的是按照访问阈值排序点的时候，分清 u 和 $w[u]$ ；排序查询的时候，是 $\text{sort}(qr + 1, qr + 1 + q, \dots)$ ，而不是 $qr + 1 + n$ 。

提交代码

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N=5e5+7;
vector<int> g[N];
int w[N];
int a[N];
int fa[N];
int sz[N];
int isv[N];

int find(int x)
{
    if(fa[x]==x)return x;
    fa[x]=find(fa[x]);
    return fa[x];
}

void merge(int i,int j)
{
    int ii=find(i);
    int jj=find(j);
    if(ii==jj)return;
    if(sz[ii]>sz[jj])
    {
        fa[jj]=ii;
        sz[ii]+=sz[jj];
    }
    else
    {
        fa[ii]=jj;
        sz[jj]+=sz[ii];
    }
}

struct query
{
    int u,x;
    int i;
    int ans;
};
query qr[N];

signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    cout.tie(0);

    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++)cin>>w[i];
    for(int i=1;i<=n;i++)
    {
        a[i]=i;
        fa[i]=i;
        sz[i]=1;
    }
    sort(a+1,a+1+n,[](const int&l,const int &r)
```

```

return w[l]<w[r];
});
//for(int i=1;i<=n;i++)cout<<a[i]<<endl;

while(m--)
{
int u,v;
cin>>u>>v;
g[u].push_back(v);
g[v].push_back(u);
}
int q;
cin>>q;
for(int i=1;i<=q;i++)
{
qr[i].i=i;
cin>>qr[i].u>>qr[i].x;
}
sort(qr+1,qr+1+q,[](const query&l,const query&r)
{
return l.x<r.x;
});

//for(int i=1;i<=q;i++)cout<<qr[i].u<<' '<<qr[i].x<<endl;

int p=1;
for(int i=1;i<=q;i++)
{
while(p<=n&&w[a[p]]<=qr[i].x)
{
int u=a[p];
for(int v:g[u])
{
if(isv[v])
{
merge(v,u);
}
}
isv[u]=1;
//cout<<"p:"<<p<<' '<<u<<endl;
p++;
}
//cout<<"find"<<find(qr[i].u)<<endl;
//for(int i=1;i<=n;i++)cout<<fa[i]<<"\n"[i==n];
//cout<<"qr"<<qr[i].x<<endl;
qr[i].ans=sz(find(qr[i].u));
}
sort(qr+1,qr+1+q,[](const query&l,const query&r)
{
return l.i<r.i;
});
for(int i=1;i<=q;i++)cout<<qr[i].ans<<'\n';
}

```

07 / 20260729-02

评测排序

牛客 2026-07-29 状压dp / SOS dp 代码：20260729-02.cpp

题目大意

给定 n 个程序，有 m 个评测点，每个点每个程序给定是否通过、评测耗时。拒绝之后的评测都不需要跑，问如何安排评测点顺序使得总时间最小。 $m \leq 20$, $n \leq 3e5$

思路

难点：

1. 发现 $m=20$ ，往状压 DP 想。
 2. 会算 $\text{cost}[S][i]$ ，也就是 SOS DP。已知集合的价值，求每个集合对应的所有超集的价值之和。
- SOS DP，超集和、子集和。

提交代码

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define double long double
const int N=2e4+7;
const int M=1<<20;
int a[N][22];
int f[M][22];
int mk[N];
int dp[M];

signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(0);
    //cout<<pow(2,20);
    int n,m;
    cin>>n>>m;
    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=m;j++)
        {
            cin>>a[i][m-j+1];
        }
        for(int j=1;j<=m;j++)
        {
            char c;
            cin>>c;
            mk[i]<=1;
            if(c=='A')
            {
                mk[i]=1;
            }
        }
        //cout<<mk[i]<<endl;
        for(int j=1;j<=m;j++)
        {
            f[mk[i]][j]+=a[i][j];
        }
    }

    int mm=1<<m;
    for(int bit=0;bit<m;bit++)
    {
        for(int mask=0;mask<mm;mask++)
        {
            if(!(mask&(1<<bit)))
            {
                for(int i=1;i<=m;i++)
                {
                    f[mask][i]+=f[mask|(1<<bit)][i];
                }
            }
        }
    }
    //for(int i=0;i<mm;i++)
    //{
    //for(int j=1;j<=m;j++)cout<<f[i][j]<<" \n"[j==m];
    //}

    for(int i=1;i<M;i++)dp[i]=1e18;
    for(int i=0;i<mm;i++)
    {
        for(int j=0;j<m;j++)
        {
            int mask=~(1<<j);
            if(i&(1<<j))
            {
                dp[i]=min(dp[i],dp[i&mask]+f[i&mask][j+1]);
            }
        }
    }
}
```



```
//cout<<i<<' '<<(i&mask)<<' '<<j<<endl;
}
}
}
cout<<dp[mm-1]<<endl;
}
```

08 / 20260729-01

字典最小博弈

牛客 2026-07-29 博弈 代码：20260729-01.cpp

题目大意

爱丽丝和鲍勃在玩一个游戏。

最初，有一个空序列。给他们一个整数 n ，他们将构造一个长度为 n 的排列 p 。

爱丽丝先下。每次移动时，当前棋手都会从 1 到 n 之间选择一个之前没有选择过的整数，并将其附加到序列的末尾。经过恰好 n 步之后，序列就变成了 $1, 2, \dots, n$ 的排列组合 p 。

排列组合 $p=(p_1, p_2, \dots, p_n)$ 的循环移动是选择索引 i 而得到的序列。 ($1 \leq i \leq n$) 并写入 $(p_i, p_{i+1}, \dots, p_n, p_1, p_2, \dots, p_{i-1})$ 。例如， $(2, 3, 1)$ 的循环移位是 $(2, 3, 1)$ 、 $(3, 1, 2)$ 和 $(1, 2, 3)$ 。

对于一个排列组合 p ，定义 $f(p)$ 为 p 的词典最小循环移位。

爱丽丝希望在词典上最小化 $f(p)$ ，而鲍勃希望在词典上最大化 $f(p)$ 。

假设双方都以最优方式下棋，请确定所得到的排列组合 $f(p)$ 。

思路

显然这个题很有打表的套路。它很可能是个连蒙带猜的题。

于是直接用 $\min(\max(\dots))$ 打表，基本没有太多的技巧。

难道博弈题就是这种神秘的规律？

提交代码

```
#include <bits/stdc++.h>
using namespace std;
#define int long long
const int N=5e5+7;
int a[N];

//string gets(vector<int>&v)
//{
//string ans;
//int flag=0;
//for(int i:v)
//{
//if(i==1)flag=1;
//if(flag)ans=ans+char(i+'0');
//}
//for(int i:v)
//{
//if(i==1)break;
//ans=ans+char(i+'0');
//}
//return ans;
```

```

//}
//
//bool isv(vector<int>&v,int i)
//{
//for(int j:v)if(i==j)return true;
//return false;
//}
//
//string solve(int k,vector<int>&v,int n)
//{
//if(k>n)return gets(v);
//string ans;
//int fir=1;
//if(k%2)
//{
//for(int i=1;i<=n;i++)
//{
//if(isv(v,i))continue;
//v.push_back(i);
//if(fir)
//{
//fir=0;
//ans=solve(k+1,v,n);
//}
//else
//{
//string tmp=solve(k+1,v,n);
//if(ans>tmp)ans=tmp;
//}
//v.pop_back();
//}
//}
//else
//{
//for(int i=1;i<=n;i++)
//{
//if(isv(v,i))continue;
//v.push_back(i);
//if(fir)
//{
//fir=0;
//ans=solve(k+1,v,n);
//}
//else
//{
//string tmp=solve(k+1,v,n);
//if(ans<tmp)ans=tmp;
//}
//v.pop_back();
//}
//}
//return ans;
//}
//
//void out(string s)
//{
//for(char c:s)cout<<c-'0'<<' ';
//cout<<endl;
//}

signed main()
{
//for(int i=2;i<=11;i+=2)
//{
//vector<int> v;
//out(solve(1,v,i));
//}
int t;
cin>>t;
while(t--)
{
int n;
cin>>n;
if(n%2)
{
a[1]=1;
a[3]=2;
int st1=n,st2=3;

```

```
for(int i=n;i>=2;i--)
{
    if(i==3)continue;
    if(i%2)a[i]=st1--;
    else a[i]=st2++;
}
for(int i=1;i<=n;i++)cout<<a[i]<<" \n"[i==n];
}
else
{
    a[1]=1;
    int st1=n,st2=2;
    for(int i=n;i>=2;i--)
    {
        if(i%2==0)a[i]=st1--;
        else a[i]=st2++;
    }
    for(int i=1;i<=n;i++)cout<<a[i]<<" \n"[i==n];
}
}
```

END

上场前最后检查

- ☐ 先读完所有题，按预期难度与队友分工。
- ☐ 写清状态、转移、复杂度和边界，再开始敲代码。
- ☐ 检查 `long long`、数组范围、初始化、多测清空与下标。
- ☐ 图论检查非连通图、重边、自环与父边；位运算检查移位宽度。
- ☐ 提交前跑样例、自造极小数据，并复查输出格式。

GL & HF.